
A VARIABLE-LENGTH GRAY CODE FOR THE NATURAL NUMBERS

A PREPRINT

• Ezequiel López-Rubio*

Department of Computer Languages and Computer Science
University of Málaga
Bulevar Louis Pasteur, 35
29071 Málaga, Spain
ezeqlr@lcc.uma.es

July 20, 2026

ABSTRACT

The reflected binary Gray code arranges the fixed-length binary representations of the integers so that consecutive numbers differ in a single bit. Its usefulness, however, is tied to a fixed word length b , which both caps the range of representable numbers at $2^b - 1$ and wastes bits on small integers. We introduce a *variable-length Gray code* V , a total function from the natural numbers to the set of all finite binary strings, obtained by taking the reflected Gray code of $n + 1$, discarding its leading zeros, and deleting the single leading one. We prove four properties of this construction. First, V is a bijection between the natural numbers and $\{0, 1\}^*$, so it is a complete code that assigns exactly one codeword to every finite binary string, including the empty string. Second, the Levenshtein (edit) distance between the codewords of two consecutive integers is always one; within a block of equal-length codewords the step is a single-bit substitution, and at each block boundary it is a single insertion. Third, codeword length is monotone non-decreasing in the encoded number and equals $\lfloor \log_2(n + 1) \rfloor$. Fourth, the transition from $(k - 1)$ -bit to k -bit codewords occurs exactly at $n = \sum_{i=0}^{k-1} 2^i = 2^k - 1$, where $V(n) = 10^k$. The code therefore realises a Hamiltonian enumeration of $\{0, 1\}^*$ under unit edit steps while retaining a near-optimal, self-adapting length profile. We discuss its compression behaviour relative to fixed-length Gray codes and its application to the *Isal** family of instruction-set string encodings of graphs, hypergraphs, molecules and symbolic-regression expressions, in which the Levenshtein distance between encodings is used as a structural-similarity proxy.

Keywords Gray code · variable-length codes · bijective numeration · Levenshtein distance · universal integer codes · edit distance · instruction-set encodings

1 Introduction

The reflected binary Gray code is one of the most widely used non-standard numeral systems in computer science and engineering [Gray, 1953, Gilbert, 1958, Doran, 2007]. Its defining property is that the binary words assigned to two numerically consecutive integers differ in exactly one bit position. This single-bit-change property makes Gray codes indispensable wherever a physical or logical quantity is sampled while it changes: rotary and linear position encoders, analog-to-digital converters, asynchronous state machines, Karnaugh maps, and the mutation neighbourhoods of genetic algorithms all exploit it to avoid the large, transient errors that ordinary binary counting produces when several bits flip at once.

The classical Gray code is, however, a *fixed-length* object. To encode integers one must commit in advance to a word length b ; the code then represents only the 2^b integers $0, 1, \dots, 2^b - 1$, and it spends the full b bits on every one of

*Corresponding author. ITIS Software. Universidad de Málaga. C/ Arquitecto Francisco Peñalosa 18, 29010, Málaga, Spain

them, including small numbers that would need far fewer. Two limitations follow. First, the representable range is bounded, so an unbounded stream of natural numbers cannot be encoded without periodically enlarging b and, in general, re-encoding. Second, the representation is wasteful for skewed distributions in which small integers dominate, a situation common when integers denote indices, counters, offsets, or repetition factors inside a larger structured object.

In this paper we remove the fixed-length restriction while preserving the essential Gray-code spirit. We define a *variable-length Gray code*

$$V : \mathbb{N} \longrightarrow \{0, 1\}^*,$$

which maps each natural number to a finite binary string of self-adapting length. The construction is elementary. Given $n \in \mathbb{N}$, we form $m = n + 1$, write the reflected Gray code of m , delete its leading zeros, and then delete the single leading one that necessarily remains. The empty string is the image of 0; longer strings encode larger integers. Table 1 lists the first fifteen codewords, computed for illustration from a four-bit Gray code.

We establish four properties, stated informally here and proved in Section 4. (i) V is a bijection onto the set of *all* finite binary strings, so every string is a codeword of exactly one integer and the code is complete. (ii) The Levenshtein distance [Levenshtein, 1966] between the codewords of n and $n + 1$ is always one, so the enumeration $V(0), V(1), V(2), \dots$ is a single-edit walk that visits every finite binary string exactly once. (iii) Codeword length is monotone non-decreasing in n and equals $\lfloor \log_2(n + 1) \rfloor$, which is within one bit of the information-theoretic ideal. (iv) The passage from one codeword length to the next happens precisely at the numbers $2^{k+1} - 1 = \sum_{i=0}^k 2^i$, at which point the codeword is a one followed by k zeros.

Taken together, these properties characterise V as a Gray code for the countably infinite set $\{0, 1\}^*$ ordered by unit edit distance, rather than for a fixed-size hypercube ordered by unit Hamming distance. The remainder of the paper is organised as follows. Section 2 reviews Gray codes, edit distances, and variable-length integer codes. Section 3 fixes notation, recalls the standard reflected Gray code, and defines V . Section 4 proves the four theorems. Section 5 discusses compression relative to fixed-length Gray codes and the application to the Isal* encodings, and Section 6 concludes.

2 Previous Works

Gray codes. The reflected binary code was patented by Gray [1953] for pulse-code communication, although the underlying single-change ordering had been used earlier in telegraphy and puzzles. Gilbert [1958] placed it on a combinatorial footing by identifying Gray codes with Hamiltonian paths on the n -dimensional hypercube, where vertices are n -bit strings and edges join strings at Hamming distance one. The standard b -bit reflected Gray code $G(\cdot, b)$ is the bijection between $\{0, \dots, 2^b - 1\}$ and $\{0, 1\}^b$ given by $\text{gray}(m) = m \oplus \lfloor m/2 \rfloor$; it is a single Hamiltonian path of the hypercube, and consecutive codewords therefore have unit Hamming distance. Beyond the binary reflected code, a large literature studies *combinatorial* Gray codes, in which the objects of a combinatorial class (combinations, permutations, partitions, spanning trees, and many others) are listed so that successive objects differ by a minimal local change; the survey by Savage [1997] and the treatment in Knuth’s *Art of Computer Programming* [Knuth, 2011] give a comprehensive account. Our construction belongs to this tradition, but the underlying combinatorial class is the set of *all* finite binary strings and the local change is a single *edit* operation rather than a bit flip; the fixed length of the classical code is replaced by a variable one.

Edit and Hamming distances. The Hamming distance [Hamming, 1950] counts positions at which two equal-length strings differ and is the natural metric of fixed-length Gray codes. When strings may have different lengths, the appropriate metric is the Levenshtein (edit) distance [Levenshtein, 1966], the minimum number of single-character insertions, deletions, and substitutions transforming one string into another. Hamming distance is the restriction of Levenshtein distance to equal-length strings and to substitution operations only; a unit Hamming distance is therefore also a unit Levenshtein distance. This relationship is what allows a variable-length code to inherit, in a suitable sense, the single-change property of the classical Gray code.

Variable-length and universal integer codes. Encoding the natural numbers by binary strings of number-dependent length is the subject of a rich body of work on universal codes. Elias [1975] introduced the γ , δ , and ω codes, self-delimiting representations whose length grows like $\log_2 n$ up to lower-order terms. Golomb [1966] gave run-length codes optimal for geometric distributions, later specialised as Rice codes. Apostolico and Fraenkel [1987] and Fraenkel and Klein [1996] developed complete and robust universal codes based on Fibonacci (Zeckendorf) representations, prized for their resynchronisation properties. A recurring design axis in this literature is whether the code is *prefix-free* (self-delimiting) or *bijective*. Bijective numeration systems, discussed for instance by Salomaa [1981] and Knuth [2011], place the finite strings over an alphabet in one-to-one correspondence with the natural numbers; the familiar binary

instance maps n to the binary representation of $n + 1$ with its leading one removed. Our code is the Gray-coded analogue of exactly this bijection: we replace the ordinary binary representation of $n + 1$ by its reflected Gray code before removing the leading one. The novelty is that this single substitution endows the bijection with a unit-edit-distance property between consecutive integers, which the ordinary bijective binary code does not possess. The trade-off between length optimality and the various structural guarantees of such codes is treated from an information-theoretic standpoint by Cover and Thomas [2006].

Instruction-set string encodings. A recent line of work encodes structured mathematical objects as strings over a small instruction alphabet that is executed by a minimal virtual machine, so that every string decodes to a valid object. This *Isal** (Instruction Set And Language) family includes *IsalProgram* for assembly-like programs [López-Rubio, 2026], *IsalGraph* for finite simple graphs [López-Rubio and Pascual-Gonzalez, 2026a], *IsalHG* for hypergraphs [Pascual-Gonzalez and López-Rubio, 2026], an instruction set and language for symbolic-regression expressions [López-Rubio and Pascual-Gonzalez, 2026b], and *IsalChem* for molecules [Thurnhofer-Hemsi et al., 2025], the last offering an alternative to the SMILES notation [Weininger, 1988]. A distinctive empirical finding of this family, reported for *IsalGraph* [López-Rubio and Pascual-Gonzalez, 2026a], is that the Levenshtein distance between two instruction strings correlates strongly with the structural (graph edit) distance between the objects they encode. Because these instruction streams contain embedded integer operands, a variable-length code whose consecutive codewords are one edit apart is directly relevant to them; we return to this point in Section 5.

3 Methodology

3.1 Notation

Let $\mathbb{N} = \{0, 1, 2, \dots\}$ be the natural numbers including zero. Let $\{0, 1\}$ be the binary alphabet and $\{0, 1\}^*$ the set of all finite strings over $\{0, 1\}$, including the empty string ε . For a string w , $L(w)$ denotes its length; $L(\varepsilon) = 0$. We write 0^k for the string consisting of k zeros and use juxtaposition for concatenation, so that 10^k is a one followed by k zeros. For an integer $m \geq 1$, $\lfloor \log_2 m \rfloor + 1$ is its number of bits.

For two strings $u, v \in \{0, 1\}^*$, the *Levenshtein distance* $d_L(u, v)$ is the minimum number of single-character insertions, deletions, and substitutions needed to transform u into v [Levenshtein, 1966]. When $L(u) = L(v)$, the *Hamming distance* $d_H(u, v)$ is the number of positions at which they differ [Hamming, 1950]. For equal-length strings, $d_L(u, v) \leq d_H(u, v)$, since every mismatch may be repaired by one substitution; in particular $d_H(u, v) = 1$ implies $d_L(u, v) = 1$.

3.2 The Standard Fixed-Length Gray Code

Fix a word length $b \geq 1$. For $0 \leq m < 2^b$, let $\beta_{b-1}\beta_{b-2}\dots\beta_0$ be the b -bit binary representation of m , with β_{b-1} the most significant bit. The b -bit *reflected binary Gray code* of m is the string $G(m, b) = \gamma_{b-1}\gamma_{b-2}\dots\gamma_0$ with

$$\gamma_{b-1} = \beta_{b-1}, \quad \gamma_i = \beta_{i+1} \oplus \beta_i \quad (0 \leq i \leq b-2), \quad (1)$$

where $\oplus$ is exclusive or. Equivalently, as an integer, $\text{gray}(m) = m \oplus \lfloor m/2 \rfloor$, and $G(m, b)$ is the b -bit binary representation of $\text{gray}(m)$. We record the standard facts we shall use [Gilbert, 1958, Doran, 2007, Knuth, 2011].

Proposition 3.1 (Basic Gray-code facts). *Let $b \geq 1$.*

- (P1) (Bijection.) *The map $m \mapsto G(m, b)$ is a bijection from $\{0, \dots, 2^b - 1\}$ onto $\{0, 1\}^b$.*
- (P2) (Adjacency.) *For $0 \leq m < 2^b - 1$, the strings $G(m, b)$ and $G(m + 1, b)$ differ in exactly one position: $d_H(G(m, b), G(m + 1, b)) = 1$.*
- (P3) (Leading bit.) *The most significant bit of $G(m, b)$ equals that of m ; hence it is 1 if and only if $m \geq 2^{b-1}$.*
- (P4) (Widening.) *If $b' > b$ and $0 \leq m < 2^b$, then $G(m, b') = 0^{b'-b} G(m, b)$.*

Proof. (P1) and (P2) are the classical characterisation of the reflected code as a Hamiltonian path of the hypercube [Gilbert, 1958]. (P3) is immediate from (1), since $\gamma_{b-1} = \beta_{b-1}$. For (P4), if $m < 2^b$ then in a b' -bit representation the bits $\beta_b, \dots, \beta_{b'-1}$ are all zero, so by (1) the Gray bits $\gamma_{b'-1} = \beta_{b'-1} = 0$ and $\gamma_i = \beta_{i+1} \oplus \beta_i = 0$ for $b \leq i \leq b' - 2$, while the bits $\gamma_{b-1}, \dots, \gamma_0$ coincide with those of $G(m, b)$ because $\gamma_{b-1} = \beta_{b-1}$ in both widths. Thus the extra high-order positions are zeros and the low-order block is unchanged. $\square$

Table 1: The first fifteen codewords of the variable-length Gray code V , computed for illustration from the four-bit reflected Gray code $G(n+1, 4)$. Leading zeros and the first one of $G(n+1, 4)$ are removed to obtain $V(n)$. The all-zero word 0000 encodes no natural number. Horizontal rules mark the boundaries between codeword-length blocks.

n	$G(n+1, 4)$	$V(n)$	n	$G(n+1, 4)$	$V(n)$
—	0000	—	7	1100	100
0	0001	ε	8	1101	101
1	0011	1	9	1111	111
2	0010	0	10	1110	110
3	0110	10	11	1010	010
4	0111	11	12	1011	011
5	0101	01	13	1001	001
6	0100	00	14	1000	000

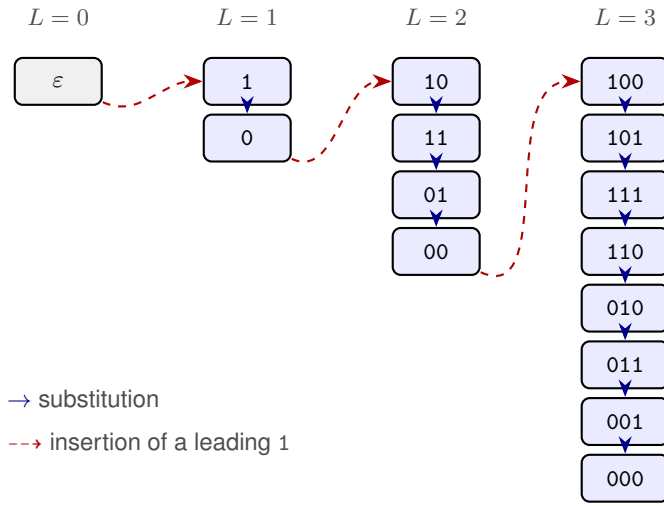


Figure 1: The enumeration $V(0), V(1), V(2), \dots$ as a walk on $\{0, 1\}^*$ under unit Levenshtein steps. Each column is a length block; within a column, consecutive codewords differ by one substitution (Theorem 4.2, interior case), and each block boundary is a single insertion of a leading one, taking 0^k to 10^k (Theorems 4.2 and 4.4). The walk visits every finite binary string exactly once (Theorem 4.1).

3.3 The Variable-Length Gray Code

Property (P4) says that the Gray code of a number is stable, up to leading zeros, under the choice of word length, provided the length is large enough to hold the number. This lets us strip the length parameter away.

Definition 3.1 (Canonical Gray word). For an integer $m \geq 1$, the *canonical Gray word* $\gamma(m) \in \{0, 1\}^*$ is $G(m, b)$ with all leading zeros removed, where $b \geq \lfloor \log_2 m \rfloor + 1$ is any word length sufficient to represent m .

By (P4) the value of $\gamma(m)$ does not depend on the choice of b , so γ is well defined. Because the most significant bit of m is 1, (P3) implies that the first symbol of $\gamma(m)$ is 1; and since removing leading zeros leaves the bits from the most significant one downward, $L(\gamma(m)) = \lfloor \log_2 m \rfloor + 1$, the bit-length of m .

Definition 3.2 (Variable-length Gray code). For $n \in \mathbb{N}$, let $m = n + 1$. The *variable-length Gray code* of n is the string $V(n) \in \{0, 1\}^*$ obtained from the canonical Gray word $\gamma(m)$ by deleting its first symbol (which is a one):

$$V(n) = \text{the string } \gamma(n+1) \text{ with its leading one removed.}$$

Equivalently, $V(n)$ is the reflected Gray code of $n+1$ with all leading zeros and then the first one removed, as in the informal statement of Section 1. The subtraction of the all-zero Gray word is built in: since $m = n+1 \geq 1$, the word $\gamma(m)$ is never empty, so the deletion of a leading one is always possible, and no natural number is mapped through the all-zero code.

4 Properties

Throughout this section, for $k \geq 0$ we call

$$B_k = \{n \in \mathbb{N} : L(V(n)) = k\}$$

the k -th length block. We first pin down the length of a codeword, which determines the blocks explicitly, and then prove the four theorems.

Lemma 4.1 (Codeword length). *For every $n \in \mathbb{N}$, $L(V(n)) = \lfloor \log_2(n+1) \rfloor$. Consequently*

$$B_k = \{n \in \mathbb{N} : 2^k - 1 \leq n \leq 2^{k+1} - 2\}, \quad |B_k| = 2^k,$$

and, equivalently in terms of $m = n + 1$, $n \in B_k$ iff $2^k \leq m \leq 2^{k+1} - 1$.

Proof. Let $m = n + 1 \geq 1$. By the remark after Definition 3.1, $L(\gamma(m)) = \lfloor \log_2 m \rfloor + 1$, and $V(n)$ deletes one symbol, so $L(V(n)) = \lfloor \log_2 m \rfloor = \lfloor \log_2(n+1) \rfloor$. Now $L(V(n)) = k$ iff $\lfloor \log_2(n+1) \rfloor = k$ iff $2^k \leq n+1 < 2^{k+1}$ iff $2^k - 1 \leq n \leq 2^{k+1} - 2$. The block has $(2^{k+1} - 2) - (2^k - 1) + 1 = 2^k$ elements. $\square$

4.1 Bijectivity

Lemma 4.2 (Block bijection). *For every $k \geq 0$, the restriction of V to B_k is a bijection onto $\{0, 1\}^k$.*

Proof. Fix $k \geq 0$ and take $b = k + 1$, which is enough bits for every m with $n \in B_k$, since by Lemma 4.1 such m satisfy $2^k \leq m \leq 2^{k+1} - 1 < 2^{k+1}$. For these m we have $m \geq 2^k = 2^{b-1}$, so by (P3) the most significant bit of $G(m, k+1)$ is 1; hence $G(m, k+1)$ has no leading zeros, $\gamma(m) = G(m, k+1)$, and

$$V(n) = \text{the last } k \text{ symbols of } G(m, k+1).$$

Consider the map $\phi : m \mapsto G(m, k+1)$. By (P1) it is a bijection from $\{0, \dots, 2^{k+1} - 1\}$ onto $\{0, 1\}^{k+1}$. Restricting its domain to $\{m : 2^k \leq m \leq 2^{k+1} - 1\} = \{m : \text{MSB of } m \text{ is } 1\}$, property (P3) shows the image is exactly the set of $(k+1)$ -bit strings whose first symbol is 1, a set of 2^k strings. The map that deletes this leading one is a bijection from those strings onto $\{0, 1\}^k$. Composing, and using $n = m - 1$, the map $n \mapsto V(n)$ is a bijection from B_k onto $\{0, 1\}^k$. $\square$

Theorem 4.1 (Bijectivity). *$V : \mathbb{N} \rightarrow \{0, 1\}^*$ is a bijection. Equivalently, every finite binary string is the codeword of exactly one natural number, the empty string being the codeword of 0.*

Proof. The blocks $\{B_k\}_{k \geq 0}$ partition $\mathbb{N}$, because by Lemma 4.1 each n lies in B_k for the unique $k = \lfloor \log_2(n+1) \rfloor$. The sets $\{\{0, 1\}^k\}_{k \geq 0}$ partition $\{0, 1\}^*$ by length. By Lemma 4.2, V maps each B_k bijectively onto $\{0, 1\}^k$. A map that bijects the blocks of one partition onto the corresponding blocks of another is itself a bijection between the underlying sets; hence $V : \mathbb{N} \rightarrow \{0, 1\}^*$ is a bijection. For $k = 0$ we have $B_0 = \{0\}$ and $\{0, 1\}^0 = \{\varepsilon\}$, so $V(0) = \varepsilon$. $\square$

4.2 Unit Levenshtein Distance Between Consecutive Codewords

We isolate the two codewords that sit at the ends of each block.

Lemma 4.3 (Block-end codewords). *For every $k \geq 0$,*

$$V(2^{k+1} - 2) = 0^k \quad \text{and} \quad V(2^{k+1} - 1) = 10^k.$$

Proof. For the first identity, let $n = 2^{k+1} - 2$, so $m = n + 1 = 2^{k+1} - 1$, whose binary representation is $k+1$ ones. Then

$$\text{gray}(m) = m \oplus \lfloor m/2 \rfloor = (2^{k+1} - 1) \oplus (2^k - 1).$$

In binary, $2^{k+1} - 1$ has ones in positions $0, \dots, k$ and $2^k - 1$ has ones in positions $0, \dots, k-1$; their exclusive or has a one in position k and zeros elsewhere, i.e. $\text{gray}(m) = 2^k$, whose canonical word is $\gamma(m) = 10^k$. Deleting the leading one gives $V(n) = 0^k$.

For the second identity, let $n = 2^{k+1} - 1$, so $m = n + 1 = 2^{k+1}$, whose binary representation is a one in position $k+1$ and zeros elsewhere. Then

$$\text{gray}(m) = 2^{k+1} \oplus 2^k,$$

which has ones in positions $k+1$ and k and zeros elsewhere, so $\gamma(m) = 110^k$. Deleting the leading one gives $V(n) = 10^k$. $\square$

Theorem 4.2 (Unit Levenshtein distance). *For every $n \in \mathbb{N}$, $d_L(V(n), V(n+1)) = 1$.*

Proof. By Lemma 4.1, n is either interior to a block or at its right end. We treat the two cases.

Interior case: n and $n+1$ lie in the same block B_k . This holds precisely when $n \neq 2^{k+1} - 2$ for the block index k of n . Put $m = n+1$; then m and $m+1$ both satisfy $2^k \leq m$, $m+1 \leq 2^{k+1} - 1$, so by the argument in Lemma 4.2 both $G(m, k+1)$ and $G(m+1, k+1)$ begin with a one and have length $k+1$, and

$$V(n) = \text{last } k \text{ symbols of } G(m, k+1), \quad V(n+1) = \text{last } k \text{ symbols of } G(m+1, k+1).$$

By (P2), $G(m, k+1)$ and $G(m+1, k+1)$ differ in exactly one position. Since both leading symbols equal 1, that position is not the leading one, so it lies among the last k positions. Deleting the common leading one therefore yields two strings of equal length k that differ in exactly one position: $d_H(V(n), V(n+1)) = 1$. Hence $d_L(V(n), V(n+1)) \leq 1$, and the value is not 0 because V is injective (Theorem 4.1); thus it equals 1.

Boundary case: $n = 2^{k+1} - 2$ and $n+1 = 2^{k+1} - 1$ for some $k \geq 0$. By Lemma 4.3, $V(n) = 0^k$ and $V(n+1) = 10^k$. Inserting a single symbol 1 at the front of 0^k produces 10^k , so $d_L(V(n), V(n+1)) \leq 1$; the two strings have different lengths and so are distinct, giving $d_L \geq 1$. Hence the distance is 1. $\square$

Remark 4.1. The proof shows more than the stated bound: consecutive codewords within a block are one *substitution* apart (equal length), whereas each block boundary is a single *insertion* of a leading one. The enumeration $V(0), V(1), \dots$ is thus a Hamiltonian walk of $\{0, 1\}^*$ under unit edit steps that alternates long runs of substitutions with a solitary insertion at each power-of-two boundary; Figure 1 depicts the first four blocks.

4.3 Monotone Codeword Length

Theorem 4.3 (Monotone length). *$L(V(n))$ is monotone non-decreasing: if $n \leq m$ then $L(V(n)) \leq L(V(m))$.*

Proof. By Lemma 4.1, $L(V(n)) = \lfloor \log_2(n+1) \rfloor$. The map $n \mapsto \log_2(n+1)$ is non-decreasing on $\mathbb{N}$, and the floor of a non-decreasing function is non-decreasing; hence $n \leq m$ implies $\lfloor \log_2(n+1) \rfloor \leq \lfloor \log_2(m+1) \rfloor$, that is $L(V(n)) \leq L(V(m))$. $\square$

4.4 The Length-Increment Step

Theorem 4.4 (Length-increment step). *For every $k \geq 0$,*

$$V\left(\sum_{i=0}^k 2^i\right) = V(2^{k+1} - 1) = 10^k.$$

Proof. The finite geometric sum equals $\sum_{i=0}^k 2^i = 2^{k+1} - 1$, so the two arguments coincide, and the value 10^k is the second identity of Lemma 4.3. $\square$

Corollary 4.1 (Block structure). *For each $k \geq 1$, the codewords of length k are exactly $\{V(n) : 2^k - 1 \leq n \leq 2^{k+1} - 2\} = \{0, 1\}^k$; the smallest such n , namely $n = 2^k - 1 = \sum_{i=0}^{k-1} 2^i$, has codeword $V(2^k - 1) = 10^{k-1}$, and the largest, $n = 2^{k+1} - 2$, has codeword $V(2^{k+1} - 2) = 0^k$.*

Proof. Immediate from Lemmas 4.1, 4.2 and 4.3, taking $k \mapsto k-1$ in Theorem 4.4 for the smallest element. $\square$

Theorem 4.4 identifies the exact moment the code lengthens: the first codeword of length k appears at $n = 2^k - 1$ and equals 10^{k-1} , and Theorem 4.2 guarantees that reaching it from the last $(k-1)$ -bit codeword 0^{k-1} costs a single insertion.

5 Discussion

5.1 Interpretation

Theorems 4.1–4.4 together say that V is a Gray code for the countably infinite object $\{0, 1\}^*$, ordered by the natural numbers, in which the local move is a unit edit rather than a unit bit flip. Classical reflected Gray codes are Hamiltonian paths of a finite hypercube under Hamming adjacency [Gilbert, 1958]; V is a Hamiltonian walk of the infinite string

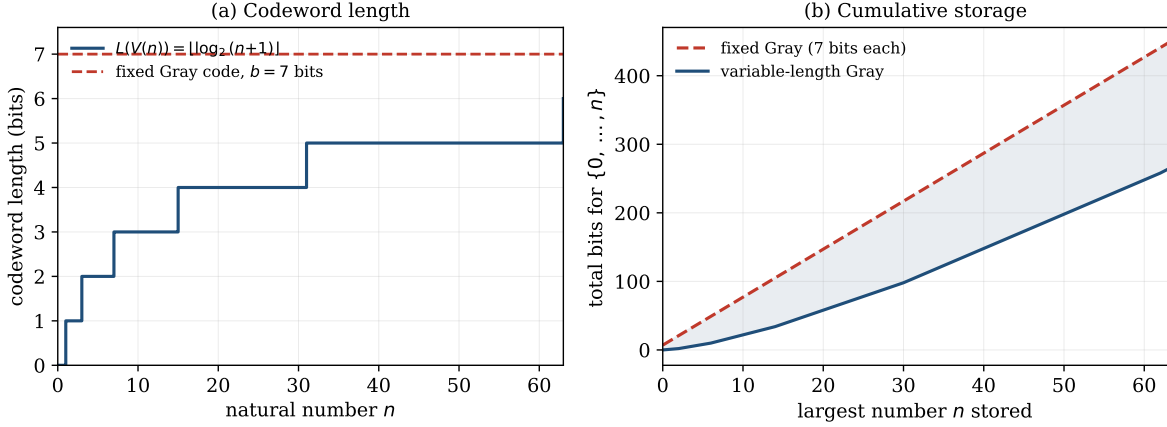


Figure 2: Compression of the variable-length Gray code V against a fixed-length Gray code sized to cover the same range ($b = 7$ bits for $N = 63$). (a) Per-integer codeword length: $L(V(n)) = \lfloor \log_2(n+1) \rfloor$ (steps) versus the constant fixed length (dashed). (b) Cumulative bits required to store $\{0, \dots, n\}$; the shaded gap is the saving, which is greatest on the small integers.

space under Levenshtein adjacency, decomposing into hypercube-like substitution runs (the length blocks) that are stitched together by single insertions at the powers of two. The construction achieves this without sacrificing bijectivity: unlike a fixed-length code, which excludes the all-zero word and caps the range at $2^b - 1$, the variable-length code uses every finite string, including the empty one, exactly once.

5.2 Compression Relative to Fixed-Length Gray Codes

Because $L(V(n)) = \lfloor \log_2(n+1) \rfloor$ (Lemma 4.1), the code spends only as many bits as the magnitude of the integer warrants, whereas a fixed b -bit Gray code spends b bits on every integer it can represent and can represent only $\{0, \dots, 2^b - 1\}$. To store all integers in $\{0, 1, \dots, N\}$, a fixed Gray code must choose $b = \lceil \log_2(N+2) \rceil$ (one extra unit because the all-zero word is unavailable), for a total of $(N+1)b$ bits, while V uses

$$\sum_{n=0}^N L(V(n)) = \sum_{n=0}^N \lfloor \log_2(n+1) \rfloor = (N+1) \lfloor \log_2(N+1) \rfloor - (2^{\lfloor \log_2(N+1) \rfloor + 1} - 2),$$

which is smaller and, per integer, is within one bit of the ideal $\log_2(n+1)$. Figure 2 contrasts the two length profiles and the cumulative storage. The saving is largest exactly where it is most often needed, on the small integers that dominate index-like and counter-like data; it is bought at the usual price of a variable-length code, namely that codewords are not, on their own, self-delimiting. Being a bijective rather than a prefix-free code, V places every finite string in use, so a boundary between adjacent codewords in a concatenated stream must be supplied by context or by composition with a self-delimiting envelope such as an Elias code [Elias, 1975] or a Fibonacci-based code [Apostolico and Fraenkel, 1987, Fraenkel and Klein, 1996]; in exchange it attains the shortest possible length profile that a monotone complete code can have, since the 2^k strings of length k are all used before any string of length $k+1$.

5.3 Application to the Isal* Instruction-Set Encodings

The property that most sharply distinguishes V from the ordinary bijective binary code, unit Levenshtein distance between consecutive integers (Theorem 4.2), is precisely the property exploited by the Isal* family of instruction-set string encodings [López-Rubio, 2026, López-Rubio and Pascual-Gonzalez, 2026a, Pascual-Gonzalez and López-Rubio, 2026, López-Rubio and Pascual-Gonzalez, 2026b, Thurnhofer-Hemsi et al., 2025]. In these methods a structured object, a program, a graph, a hypergraph, a symbolic-regression expression, or a molecule, is written as a string over a small instruction alphabet executed by a minimal virtual machine, and structural similarity between objects is estimated by the Levenshtein distance between their strings. For IsalGraph this correspondence is quantified: the edit distance between instruction strings correlates strongly with the graph edit distance between the encoded graphs [López-Rubio and Pascual-Gonzalez, 2026a].

Instruction streams of this kind embed integer operands, node and pointer indices, arity or repetition counts, offsets into the circular doubly linked list on which these machines operate. If such operands are written with V , then two objects

that differ by an operand changing from n to $n \pm 1$ differ, in that field, by a single edit (Theorem 4.2), so a small change in the object induces a small change in the string. This aligns the operand encoding with the metric-preservation goal that motivates the Isal* framework, and it does so while keeping operands compact through the monotone, near-logarithmic length profile of Theorem 4.3. A natural programme of *future work* is therefore twofold:

- (a) quantify the compression obtained by replacing fixed-width operand fields in Isal* streams, and in fixed-length Gray-coded data generally, with V , along the lines of Figure 2; and
- (b) integrate V into the Isal* encoders [López-Rubio, 2026, López-Rubio and Pascual-Gonzalez, 2026a, Pascual-Gonzalez and López-Rubio, 2026, López-Rubio and Pascual-Gonzalez, 2026b, Thurnhofer-Hemsi et al., 2025] and measure the effect on the correlation between string edit distance and structural distance, and on downstream language-model processing of the encodings.

Because V is bijective, every operand field remains valid under arbitrary edits, matching the closed-alphabet design principle of the Isal* machines, in which every string decodes to a valid object. This compatibility suggests that the variable-length Gray code is a suitable numeric primitive for that family of encodings.

6 Conclusions

We have introduced a variable-length Gray code V that maps the natural numbers onto the set of all finite binary strings by taking the reflected Gray code of $n + 1$, stripping its leading zeros, and deleting the leading one. We proved four properties. The code is a bijection onto $\{0, 1\}^*$, hence a complete code that uses every string exactly once, including the empty string (Theorem 4.1). Consecutive integers have codewords at unit Levenshtein distance, a single substitution inside each length block and a single insertion at every power-of-two boundary, so the enumeration is a Hamiltonian walk of the string space under unit edit steps (Theorem 4.2). Codeword length is monotone non-decreasing and equals $\lfloor \log_2(n + 1) \rfloor$ (Theorem 4.3). The code lengthens exactly at $n = \sum_{i=0}^k 2^i = 2^{k+1} - 1$, where the codeword is 10^k (Theorem 4.4).

The result is a code that keeps the single-change spirit of the classical Gray code while lifting its fixed-length restriction, gaining an unbounded range, a near-optimal self-adapting length, and completeness over $\{0, 1\}^*$. These features make V attractive both as a stand-alone variable-length numeral system and as a numeric primitive inside the Isal* instruction-set encodings, where the correspondence between edit distance and structural distance is central. Quantifying the compression it offers over fixed-length Gray codes, and integrating it into the Isal* encoders to measure its effect on similarity preservation and on downstream learning, are the immediate next steps.

References

- Frank Gray. Pulse code communication, 1953. U.S. Patent 2,632,058, filed 13 Nov. 1947, granted 17 Mar. 1953.
- Edgar N. Gilbert. Gray codes and paths on the n -cube. *Bell System Technical Journal*, 37(3):815–826, 1958. doi:10.1002/j.1538-7305.1958.tb03887.x.
- Robert W. Doran. The Gray code. Technical Report CDMTCS-304, Centre for Discrete Mathematics and Theoretical Computer Science, University of Auckland, 2007.
- Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966.
- Carla Savage. A survey of combinatorial Gray codes. *SIAM Review*, 39(4):605–629, 1997. doi:10.1137/S0036144595295272.
- Donald E. Knuth. *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms, Part 1*. Addison-Wesley, Upper Saddle River, NJ, 2011.
- Richard W. Hamming. Error detecting and error correcting codes. *Bell System Technical Journal*, 29(2):147–160, 1950. doi:10.1002/j.1538-7305.1950.tb00463.x.
- Peter Elias. Universal codeword sets and representations of the integers. *IEEE Transactions on Information Theory*, 21(2):194–203, 1975. doi:10.1109/TIT.1975.1055349.
- Solomon W. Golomb. Run-length encodings. *IEEE Transactions on Information Theory*, 12(3):399–401, 1966. doi:10.1109/TIT.1966.1053907.
- Alberto Apostolico and Aviezri S. Fraenkel. Robust transmission of unbounded strings using Fibonacci representations. *IEEE Transactions on Information Theory*, 33(2):238–245, 1987. doi:10.1109/TIT.1987.1057284.

- Aviezri S. Fraenkel and Shmuel T. Klein. Robust universal complete codes for transmission and compression. *Discrete Applied Mathematics*, 64(1):31–55, 1996. doi:10.1016/0166-218X(93)00116-H.
- Arto Salomaa. *Jewels of Formal Language Theory*. Computer Science Press, Rockville, MD, 1981.
- Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, Hoboken, NJ, 2nd edition, 2006.
- Ezequiel López-Rubio. The IsalProgram programming language, 2026. URL <https://arxiv.org/abs/2605.17008>.
- Ezequiel López-Rubio and Mario Pascual-Gonzalez. Instruction set for the representation of graphs, 2026a. URL <https://arxiv.org/abs/2603.11039>.
- Mario Pascual-Gonzalez and Ezequiel López-Rubio. Instruction set and language for hypergraphs, 2026. URL <https://arxiv.org/abs/2607.10194>.
- Ezequiel López-Rubio and Mario Pascual-Gonzalez. Instruction set and language for symbolic regression, 2026b. URL <https://arxiv.org/abs/2603.21836>.
- Karl Thurnhofer-Hemsi, Iván García-Aguilar, José David Fernández-Rodríguez, and Ezequiel López-Rubio. Representation of molecules by sequences of instructions. *Journal of Chemical Information and Modeling*, 65(15):7936–7955, 2025. doi:10.1021/acs.jcim.5c00354.
- David Weininger. SMILES, a chemical language and information system. 1. introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988. doi:10.1021/ci00057a005.